CHUYÊN ĐỀ · MÔN LẬP TRÌNH ĐA NỀN TẢNG

Quản lý trạng thái với Provider và ChangeNotifier trong Flutter

Ứng dụng thực tế Course Registration & Phòng thí nghiệm Provider Lab

Thành viên **Đặng Công Thắng** · Lớp 23KTMT2 · MSSV: 106230061

Võ Văn Thái · Lớp 23KTMT1 · MSSV: 106230027

Giảng viên Nguyễn Duy Nhật Viễn

Trường Đại học Bách khoa – Đại học Đà Nẵng · Khoa Điện tử Viễn thông

Mã nguồn công khai: `github.com/cityengin/course-registration`

Phân công

Đặng Công Thắng	52%

- Model `Course`, danh mục 10 môn
- `RegistrationNotifier` (14–24 TC)
- 2 màn hình + 3 widget con
- Demo Panel, kiểm thử, slide

Võ Văn Thái	48%

- Màn `ProviderLabScreen`
- `LabNotifier` (Provider cục bộ)
- `RebuildTracker` + `BuildCountChip`
- So sánh `setState` và 4 cách đọc

Tổng quan đề tài và kiến trúc giải pháp

Vấn đề: Cùng một dữ liệu hiển thị ở 4 nơi. Chỉ dùng `setState` → phải truyền qua nhiều tầng và dựng lại cả cây widget.

Course Registration

Ứng dụng đăng ký học phần

- 10 học phần mẫu (2–4 TC)
- Ràng buộc 14–24 tín chỉ
- Đồng bộ giữa 2 màn hình

Demo Panel

Bảng điều khiển demo

- 4 kịch bản thao tác nhanh
- Nhật ký `notifyListeners`
- Notify có điều kiện

Provider Lab

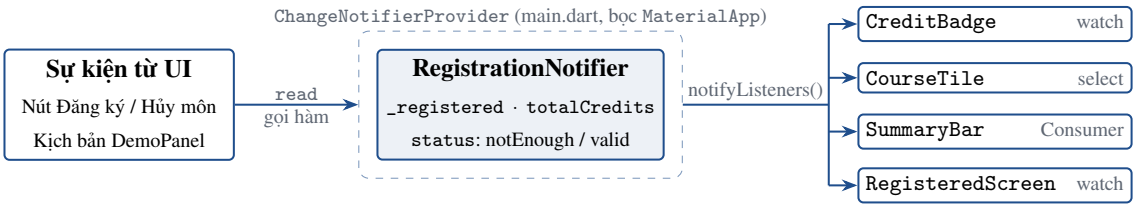
Phòng thí nghiệm rebuild

- `setState` vs Provider
- 4 cách đọc state
- Chip đếm `Build: n`

Flutter (Material 3) · `provider 6.1.5` · 7 test tự động (5 unit + 2 widget)

Kiến trúc phân lớp và luồng dữ liệu một chiều

Sự kiện UI → Notifier đổi state → `notifyListeners()` → widget lắng nghe rebuild.



Giao diện (UI) `views/ widgets/`

Chỉ hiển thị và phát sự kiện.

Trạng thái (State) `notifiers/`

Nghiệp vụ + quyết định khi nào notify.

Dữ liệu (Data) `models/ data/`

Model `Course` bất biến, 10 môn mẫu.

RegistrationNotifier: ChangeNotifier chứa toàn bộ nghiệp vụ

- **Đóng gói:** `_registered` private, chỉ đọc qua getter.
- **Dẫn xuất:** `totalCredits`, `status` tính từ danh sách.
- **Ràng buộc:** 14–24 TC, không trùng môn.
- **Lỗi → không notify:** `register()` trả `String?`.

lib/notifiers/registration_notifier.dart **(rút gọn)**

```dart
class RegistrationNotifier extends ChangeNotifier {
  static const int minCredits = 14;
  static const int maxCredits = 24;
  final List<Course> _registered = [];
  List<Course> get registered =>
      List.unmodifiable(_registered);
  int get totalCredits => _registered.fold(
      0, (sum, course) => sum + course.credits);
  String? register(Course course) {
    if (isRegistered(course.id)) return 'Đã đăng ký!';
    if (totalCredits + course.credits > maxCredits) {
      return 'Vượt quá giới hạn tối đa (24 tín chỉ)!';
    }
    _registered.add(course);
    notifyListeners(); // chỉ khi state thật sự đổi
    return null;
  }
}
```

Cung cấp state: ChangeNotifierProvider toàn cục và cục bộ

lib/main.dart

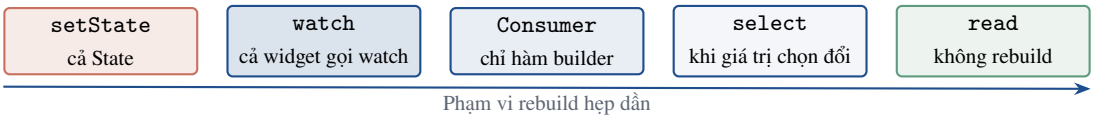
```dart
// Bọc ngoài MaterialApp: mọi màn hình dùng chung
ChangeNotifierProvider(
  create: (_) => RegistrationNotifier(),
  child: MaterialApp(
    home: const CourseListScreen(),
  ),
)
```

lib/lab/provider_lab_screen.dart

```dart
// Chỉ tồn tại trong màn Provider Lab
ChangeNotifierProvider(
  create: (_) => LabNotifier(),
  child: const _ProviderLabView(),
)
// Pop màn hình -> Provider tự gọi dispose()
```

Tiêu chí	**Toàn cục** (`RegistrationNotifier`)	**Cục bộ** (`LabNotifier`)
Phạm vi	Mọi màn hình	Chỉ màn Lab
Vòng đời	Suốt thời gian chạy app	Tự `dispose()` khi rời Lab
Mục đích	Dữ liệu đăng ký dùng chung	Đo rebuild, tách khỏi dữ liệu thật

Bốn cách đọc state trong Provider



Cách đọc	Rebuild khi nào	Áp dụng trong dự án
`context.watch<T>()`	Mỗi lần notify	`CreditBadge`, `RegisteredScreen`
`Consumer<T>`	Chỉ chạy lại `builder`	`SummaryBar`
`context.select<T,R>()`	Khi giá trị chọn đổi	`CourseTile`
`context.read<T>()`	Không bao giờ	Nút bấm (`onPressed`)

`watch`/`select`/`Consumer` trong `build()`; `read` trong callback.

Thu hẹp phạm vi rebuild: context.select và Consumer

`lib/widgets/course_tile.dart` **(rút gọn)**

```dart
// Mỗi thẻ chỉ theo dõi trạng thái của chính môn đó
final isRegistered =
    context.select<RegistrationNotifier, bool>(
  (n) => n.isRegistered(course.id),
);
// Nút Đăng ký: read, không lắng nghe
FilledButton.icon(
  onPressed: () {
    final error = context
        .read<RegistrationNotifier>()
        .register(course);
    if (error != null) {
      // hiện SnackBar đỏ báo lỗi
    }
  },
)
```

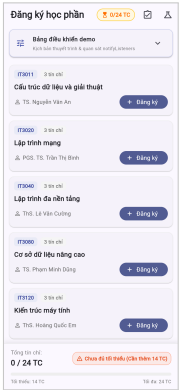
Đăng ký 1 môn: chỉ thẻ đó rebuild.

`lib/widgets/summary_bar.dart` **(rút gọn)**

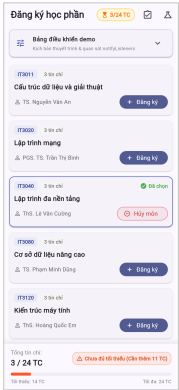
```dart
Consumer<RegistrationNotifier>(
  builder: (context, notifier, child) {
    final total = notifier.totalCredits;
    return Column(children: [
      Text('$total / 24 TC'),
      LinearProgressIndicator(
        value: (total / 24).clamp(0.0, 1.0),
      ),
      child!, // nhúng phần tĩnh
    ]);
  },
  // Dựng 1 lần, không rebuild khi notify
  child: Row(children: [
    Text('Tối thiểu: 14 TC'),
    Text('Tối đa: 24 TC'),
  ]),
)
```

Chỉ `builder` dựng lại; phần tĩnh qua `child`.

Giao diện thực tế: Một thao tác, ba vị trí cùng cập nhật



Ban đầu: 0/24 TC

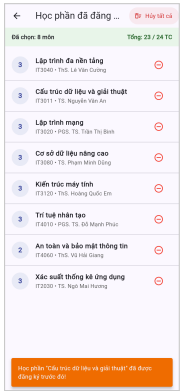


Sau khi đăng ký 1 môn

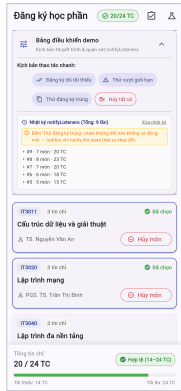
- **Thao tác:** Đăng ký môn Lập trình đa nền tảng (3 TC).
- **CreditBadge** (`watch`): 0/24 → 3/24 TC.
- **CourseTile** (`select`): viền xanh, nút "Hủy môn".
- **SummaryBar** (`Consumer`): "Cần thêm 11 TC".
- Không truyền dữ liệu thủ công.

Đồng bộ trạng thái giữa hai màn hình

- **Cùng một instance:** màn mới vẫn nằm dưới Provider gốc.
- **Hủy ở màn này, màn kia tự đổi:** 23 TC → 20 TC.
- **Không cần callback** qua `Navigator`.
- **Hủy tất cả:** xác nhận → `clearAll()`.



Màn đã đăng ký: 23 TC



Back về: tự còn 20 TC

Bảng điều khiển Demo và Nhật ký notifyListeners

`DemoPanel` gom 4 kịch bản thao tác nhanh:

1. **Đủ tối thiểu:** tự chọn đến $\geq$ 14 TC.
2. **Vượt giới hạn:** SnackBar đỏ.
3. **Đăng ký trùng:** SnackBar cam, nhật ký **không** thêm dòng.
4. **Hủy tất cả:** về 0 TC.

`lib/widgets/demo_panel.dart (_NotifyLog)` **(rút gọn)**

```dart
void initState() {
  super.initState();
  _notifier = context.read<RegistrationNotifier>();
  _notifier.addListener(_onNotified); // +1 dòng log
}
void dispose() {
  _notifier.removeListener(_onNotified); // tránh rò rỉ
  super.dispose();
}
```



Vượt giới hạn 24 TC



Trùng môn: vẫn 8 lần

Trùng môn: bộ đếm vẫn 8 → chỉ notify khi state đổi.

Provider Lab (1): setState và Provider



Ban đầu: Build 1



Bấm tăng 3 lần mỗi khối

Sau 3 lần bấm tăng ở mỗi khối:

Widget	**Khối A** `setState`	**Khối B** `LabNotifier`
Khối cha	**Build 4**	**Build 1**
Hiển thị số	**Build 4**	**Build 4** (`watch`)
Widget con tĩnh	**Build 4**	**Build 1**

- `setState`: cả `build()` chạy lại, kể cả phần không đổi.
- Provider: chỉ widget gọi `watch` rebuild.

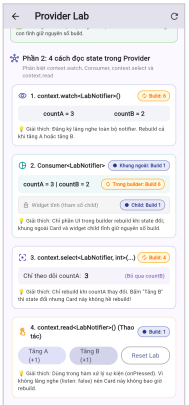
Provider Lab (2): Đo số lần rebuild của 4 cách đọc state

Sau khi `countA` tăng 3 lần và `countB` tăng 2 lần (5 lần notify):

Card	Số lần build	Giải thích
1. `context.watch`	**6**	Mỗi lần notify
2. `Consumer`	builder **6**, còn lại **1**	Chỉ `builder`
3. `select` (A)	**4**	Chỉ khi A đổi
4. `context.read`	**1**	Không lắng nghe

Cách đo:

- `RebuildTracker.hit()` đếm số lần `build()`.
- `SingleChildScrollView`: cuộn không làm sai số đếm.
- Nút reset đổi `Key` → mọi chip về Build 1.



Phần 2 sau khi Tăng B

Kết luận và Tài liệu tham khảo

Bài học kiến trúc cốt lõi

- **Tách lớp:** widget hiển thị, Notifier giữ nghiệp vụ.
- **Chọn đúng cách đọc:** `watch`, `Consumer`/`select`, `read`.
- **Notify có điều kiện:** kiểm chứng bằng `addListener`.
- **Kiểm thử:** 5 unit + 2 widget test đều đạt.

Mã nguồn & Tài liệu tham khảo

- **Mã nguồn công khai:**
  `github.com/cityengin/course-registration`
- **Tài liệu tham khảo chính thức:**
  `docs.flutter.dev/data-and-backend/state-mgmt/simple`
  `pub.dev/packages/provider`
  `api.flutter.dev/flutter/foundation/ChangeNotifier-class.html`

✓ **Cảm ơn thầy và các bạn đã lắng nghe.**